# 3. Funkcje przeciążone, funkcje inline, funkcje rekurencyjne

## 1. Funkcje przeciążone

Funkcje przeciążone to funkcje o tej samej nazwie, ale o różnych listach parametrów. Różnica może dotyczyć liczby parametrów lub ich typów. Kompilator wybiera właściwą wersję funkcji podczas kompilacji na podstawie argumentów użytych w wywołaniu.

```
int suma(int a, int b) {
    return a + b;
}

double suma(double a, double b) {
    return a + b;
}
```

Sama różnica w typie zwracanym nie wystarcza do przeciążenia funkcji. Wywołanie funkcji nie zawsze wskazuje jednoznacznie, jaki typ wyniku miałby zostać wybrany.

## 2. Zastosowanie przeciążania

Przeciążenie ma sens wtedy, gdy kilka funkcji wykonuje podobne działanie, ale dla różnych typów danych lub różnej liczby danych wejściowych. Przykładem może być funkcja obliczająca średnią dwóch albo trzech liczb.

```
double srednia(double a, double b) {
    return (a + b) / 2;
}

double srednia(double a, double b, double c) {
    return (a + b + c) / 3;
}
```

## 3. Funkcje inline

Funkcja inline to funkcja, której kod kompilator może wstawić bezpośrednio w miejscu wywołania. Dzięki temu można ograniczyć koszt wywołania bardzo krótkiej funkcji. Jest to jednak tylko sugestia - kompilator może zdecydować inaczej.

```
inline int kwadrat(int x) {
    return x * x;
}
```

Funkcje inline powinny być krótkie, proste i często wywoływane. Nie należy używać inline jako automatycznego sposobu „przyspieszania” każdego fragmentu programu.

## 4. Funkcje rekurencyjne

Funkcja rekurencyjna wywołuje samą siebie. Rekurencja jest użyteczna wtedy, gdy problem można rozbić na podobny, ale mniejszy podproblem. Każda poprawna rekurencja musi mieć warunek zakończenia.

| Element rekurencji | Znaczenie |
| --- | --- |
| przypadek bazowy | Sytuacja, w której funkcja kończy działanie bez kolejnego wywołania samej siebie. |
| krok rekurencyjny | Wywołanie tej samej funkcji dla prostszego lub mniejszego przypadku. |

## 5. Przykład rekurencji - silnia

```
int silnia(int n) {
    if (n == 0) {
        return 1;
    }
    return n * silnia(n - 1);
}
```

Dla n = 4 obliczenie przebiega według schematu: 4 * silnia(3), 3 * silnia(2), 2 * silnia(1), 1 * silnia(0). Przypadek bazowy silnia(0) zwraca 1.

## 6. Przykład rekurencji - NWD

```
int nwd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return nwd(b, a % b);
}
```

W algorytmie Euklidesa kolejne wywołania zmniejszają problem, ponieważ drugi argument staje się resztą z dzielenia. Gdy drugi argument wynosi 0, funkcja zwraca pierwszy argument jako NWD.

## 7. Rekurencja bezpośrednia i pośrednia

Rekurencja bezpośrednia występuje, gdy funkcja wywołuje samą siebie. Rekurencja pośrednia występuje wtedy, gdy funkcja wywołuje inną funkcję, a ta prowadzi do ponownego wywołania pierwszej.